RESEARCH

# Structural Recomputation Framework: A Deterministic Execution Abstraction for Memory-Constrained Dynamic Programming in Bioinformatics

MD. Arshad

Department of Computer Science, Jamia Millia Islamia, New Delhi, India

**Abstract**

Dynamic programming (DP) algorithms are foundational to bioinformatics, yet their application to genomic-scale data is frequently constrained by a quadratic memory bottleneck. This storage requirement primarily arises from the necessity of persisting traceback matrices or trellis structures to enable path reconstruction and posterior decoding. While specialized solutions such as Hirschberg's algorithm and algorithm-specific checkpointing schemes have been developed to mitigate these constraints, they often remain tightly coupled to specific recurrences and lack a unified execution abstraction. This work introduces the Structured Recomputation Framework (SRF), a deterministic execution-level abstraction that decouples algorithmic recurrence definitions from physical execution schedules. SRF employs a recurrence-agnostic design and a structured recomputation schedule to enforce strict, user-defined memory upper bounds. The framework's utility is demonstrated across diverse algorithmic paradigms, including sequence alignment (Needleman-Wunsch), Hidden Markov Model evaluation (Forward and Viterbi), and graph-based dynamic programming. Empirical validation on mitochondrial DNA and Gene Ontology datasets confirms a reduction in peak working set size from $O(N^2)$ to $O(N)$, with memory savings exceeding 99% for extreme-scale workloads. Deterministic properties are verified through cross-platform and compiler invariance checks, ensuring bit-exact reproducibility across heterogeneous compute environments. SRF provides a stable foundation for memory-efficient biological computing without requiring improvement in asymptotic time complexity, effectively trading compute cycles for spatial capacity to enable exact analyses on commodity hardware.

**Keywords:** Dynamic Programming; Sequence Alignment; Memory Constraints; Recomputation; Deterministic Execution; Numerical Constitutionalism; Scientific Provenance

## 1 Background

### 1.1 The Centrality of Dynamic Programming in Molecular Biology

Since the conceptualization of the first sequence alignment algorithms, dynamic programming (DP) has served as the mathematical foundation for computational molecular biology. The initial work by Needleman and Wunsch [1] introduced a rigorous framework for finding the optimal global alignment of two protein sequences, formalizing the biological intuition that similar sequences share a common evolutionary ancestry. This was followed by the landmark Smith-Waterman algorithm [2], which adapted the DP approach to find optimal local alignments, allowing

researchers to identify conserved domains and motifs within otherwise divergent sequences.

The utility of DP extends beyond simple pairwise alignment. It is the engine behind Hidden Markov Models (HMMs) used for gene prediction, protein family modeling, and phylogenetic analysis [3, 4]. The Viterbi and Forward algorithms, both based on DP recurrences, allow for the probabilistic interpretation of biological sequences, providing a way to model the underlying generative processes of evolution [5, 6]. Despite the rise of heuristic methods like BLAST [7, 8] and FASTA [9], DP remains the "gold standard" for accuracy and sensitivity in biological sequence comparison [10].

### 1.2 The Quadratic Memory Bottleneck in Classical DP

The fundamental challenge in applying classical DP to molecular biology is the storage requirement. For two sequences of length $N$ and $M$, the standard algorithm computes a score matrix $D(i, j)$ of size $(N + 1) \times (M + 1)$. To find the optimal alignment path, the entire matrix must be stored to perform a traceback from the final cell $D(N, M)$ to the origin $D(0, 0)$. As genome-scale sequencing becomes routine, sequence lengths $N$ and $M$ can reach millions of nucleotides, leading to a memory requirement of $O(NM)$.

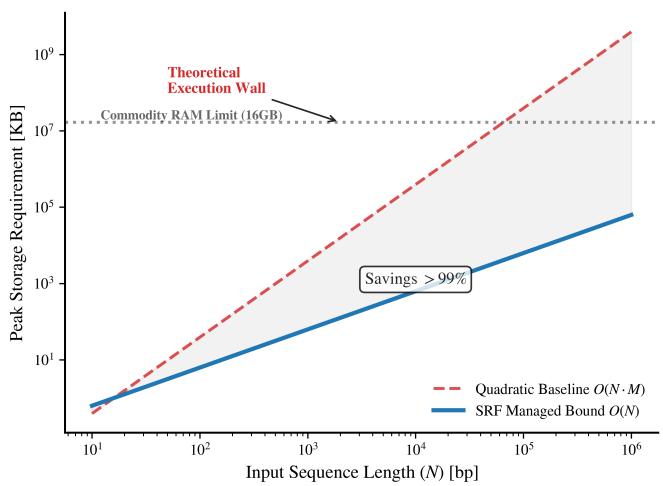


**Figure 1** Storage Complexity Divergence: Classical Quadratic Growth Barrier vs. SRF $O(N)$ Managed Boundary. The shaded region represents the physical memory savings achieved by the framework.

For a relatively small bacterial genome (e.g., $N = M = 10^6$), a single matrix using 4-byte integers requires 4 terabytes of memory, which far exceeds the capacity of standard compute nodes. This quadratic storage scaling is not merely a performance issue; it is a hard physical limit that prevents the application of exact alignment to large-scale biological data. The necessity of the traceback operation is the primary driver of this constraint, as each cell $D(i, j)$ must be available to determine the optimal move (match, mismatch, or gap) at step $(i, j)$ [11].

### 1.3 Traceback and Memory Persistence

The traceback operation is a backward traversal through the DP matrix. At each step, the algorithm chooses the neighbor that contributed to the current cell's optimal score. This requires that every cell computed during the forward pass remains persistent in memory. While the score of the optimal alignment can be computed in $O(\min(N, M))$ space by only keeping the current and previous rows (or columns) [12], the reconstruction of the alignment itself remains bound to $O(NM)$ in the naive implementation [13, 14].

### 1.4 The Hirschberg Linear-Space Algorithm

A major breakthrough in addressing the storage bottleneck was Hirschberg's algorithm [15], which uses a divide-and-conquer strategy to reduce the memory complexity to $O(\min(N, M))$. The algorithm recursively divides the alignment problem into smaller subproblems. In the forward pass, it computes the scores for a middle row $i = N/2$ from both the top-down and bottom-up (reverse) directions. By finding the index $k$ that maximizes the sum of these two scores, the algorithm identifies a point $(N/2, k)$ that must lie on the optimal path.

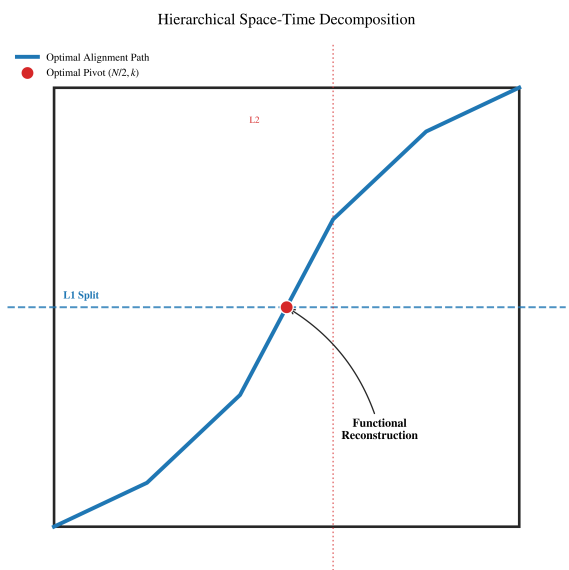


**Figure 2** Conceptual Overview of the Hirschberg Linear-Space Algorithm: Hierarchical split points and bidirectional score passes.

This approach successfully trades computation for memory. By recomputing parts of the matrix at each level of the recursion, Hirschberg's algorithm achieves a linear space bound. However, this comes at the cost of doubling the total number of operations compared to the quadratic-space version. Despite this overhead, Hirschberg's method and its later adaptations remain the primary solution for large-scale pairwise global alignment [16].

### 1.5 Myers-Miller and Affine-Gap Adaptations

Biological sequence alignment often requires affine gap penalties, where the cost of opening a gap is higher than the cost of extending it. This more realistically models

the biological process of insertions and deletions (indels). Gotoh [12] provided a quadratic-space DP solution for affine gaps by using three matrices $(M, I, J)$.

Myers and Miller [17] later adapted Hirschberg's linear-space framework to the affine gap model. Their implementation is particularly significant because it demonstrated that the linear-space benefit could be maintained even with the more complex state transitions required for affine gaps. By carefully managing the dependencies between the multiple matrices, they reduced the memory overhead while maintaining the $O(N)$ space complexity. This work is the foundation for almost all modern high-sensitivity alignment tools that handle large sequences [18].

### 1.6 Memory Scaling in Profile HMMs

The challenges of memory scaling are even more pronounced in the context of Hidden Markov Models (HMMs). Profile HMMs, popularized by tools like HMMER, are used to represent protein families and DNA motifs [6]. The Viterbi algorithm and the Forward/Backward algorithms are the standard methods for sequence-to-profile comparison.

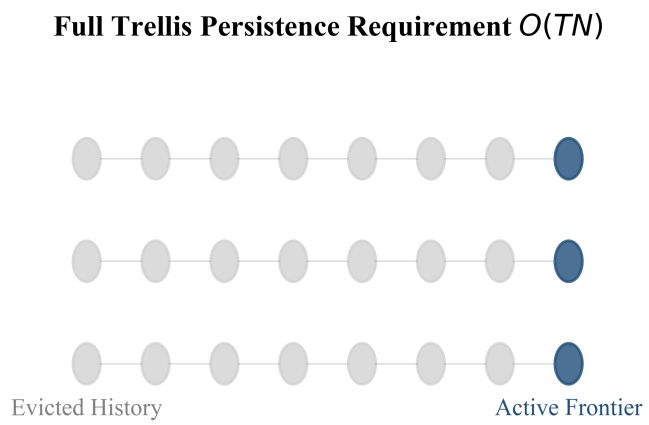


**Figure 3** HMM Trellis Persistence Model: Illustration of full trellis storage requirement versus the active frontier persistence strategy.

In an HMM with $K$ states and a sequence of length $N$, the trellis matrix has $K \times N$ cells. For complex profile HMMs with hundreds of states and sequence databases reaching gigabases in size, the storage of the trellis for posterior decoding or alignment reconstruction becomes prohibitive [19]. While simple Viterbi decoding can be performed with some memory optimizations, full posterior decoding using the Forward and Backward variables requires the entire trellis, once again leading to a storage requirement that scales linearly with the sequence length and state count, often reaching the limits of modern hardware for long reads or multi-domain proteins.

### 1.7 Forward and Viterbi: Probabilistic DP

The Viterbi algorithm finds the single most probable path through an HMM, while the Forward and Backward algorithms compute the total probability of the sequence by summing over all possible paths. Both are forms of DP and suffer from the same fundamental storage constraints for path reconstruction. The Forward algorithm, in particular, is sensitive to numerical precision, requiring the storage of high-precision floating-point values in each trellis cell, which further inflates the memory footprint compared to the integer-based scores used in sequence alignment [3].

### 1.8 Dimensional Scaling in Multiple Sequence Alignment

Multiple Sequence Alignment (MSA) represents the extreme end of the memory scaling problem. Finding the optimal alignment of $L$ sequences of average length $N$ is equivalent to finding the shortest path in an $L$-dimensional hyperspace. The complexity scales as $O(N^L)$, making exact MSA computationally and spatially impossible for even a small number of sequences [20].

While heuristic and progressive alignment methods are typically used in practice, there are cases where exact or near-exact sub-problems must be solved. In these scenarios, the memory bottleneck is the absolute limiting factor. Even with modern optimizations and high-performance computing (HPC) clusters, the storage of the $L$-dimensional matrix remains the primary barrier to exact MSA [21].

### 1.9 Cache-Aware and Cache-Oblivious DP

The performance of DP algorithms is increasingly determined by their interaction with the computer's memory hierarchy rather than the raw speed of the CPU. Modern architectures feature several layers of cache (L1, L2, L3) between the processor and the main RAM. Traditional row-by-row or column-by-column DP traversals often result in poor cache locality, leading to frequent cache misses and slow execution [21].

Cache-oblivious algorithms, introduced by Frigo et al. [22], use recursive divide-and-conquer structures to automatically optimize for any cache hierarchy without explicit parameter tuning. These techniques have been applied to DP to improve spatial and temporal locality [23]. By reorganizing the computation into blocks that fit within cache lines, these algorithms reduce the memory bandwidth requirements and significantly speed up execution on modern hardware. However, these optimizations focus primarily on runtime performance and often maintain the underlying $O(NM)$ storage requirement for the full matrix.

### 1.10 Memory Bandwidth and the "Memory Wall"

A critical limitation in modern computing is the "memory wall"—the widening gap between the speed of the CPU and the bandwidth of the main memory. Even if a DP algorithm has low computational complexity, its performance may be throttled by the time it takes to fetch data from RAM. This is especially true for large-scale DP where the matrix size exceeds the total cache capacity [21, 24].

In biological sequence analysis, this means that even if we use SIMD (Single Instruction, Multiple Data) instructions to vectorize the score calculations [25–27], the performance may still be limited by the throughput of the memory bus. This architectural reality necessitates algorithms that can trade inexpensive compute cycles

for expensive memory accesses, a strategy that is at the heart of recomputation-based frameworks.

### 1.11 Recomputation and Checkpointing Literature

Recomputation, also known as checkpointing, is a well-established technique in the field of automatic differentiation and large-scale scientific computing. Griewank [28, 29] formalized the trade-off between the number of checkpoints stored and the amount of recomputation required to evaluate gradients in reverse mode.

The Hirschberg algorithm is a specific instance of this principle applied to sequence alignment. By storing only the middle row as a "checkpoint" and recomputing the preceding rows, it achieves a significant memory reduction. More general checkpointing strategies involve storing periodic rows or columns to allow for partial tracebacks, a technique often used in HMM software and large-scale DP libraries to manage memory usage without incurring the full $O(N)$ space overhead of Hirschberg's method.

### 1.12 Formal Identification of the Research Gap

Despite the existence of specialized solutions like Hirschberg's algorithm for pairwise alignment and various checkpointing schemes for HMMs, several critical gaps remain in the literature:

1. **Problem-Specific Implementations:** Existing memory-reduction techniques are typically tightly coupled to specific algorithms (e.g., global alignment with affine gaps). There is a lack of a unified execution abstraction that can be applied to diverse DP problems without manual re-derivation of the recomputation logic.
2. **Lack of Abstraction Layers:** Most bioinformatics tools implement memory management at the same level as the biological recurrences. This leads to code duplication and prevents the application of low-level architectural optimizations (like cache-aligned tiling) across different algorithms.
3. **Lack of Cross-Algorithm Empirical Characterization:** While the theoretical benefits of recomputation are well-understood, there is a lack of comprehensive empirical studies that characterize the trade-offs between memory, recomputation, and cache locality across multiple biological DP paradigms (NW, HMM, Graph-DP) in a single unified framework.

These gaps indicate a need for a more generalized approach to memory management in biological DP—one that separates the algorithmic recurrence from the physical execution strategy.

**Table 1** DP memory reduction techniques.

| Method | Space | Time | Level |
|---|---|---|---|
| NW | $O(NM)$ | $O(NM)$ | Alg |
| Hirschberg | $O(N)$ | $2O(NM)$ | Alg |
| Myers-Miller | $O(N)$ | $2O(NM)$ | Alg |
| SRF | $O(N_{buf})$ | $O(NM) + recomp$ | Exec |

### 1.13 The Structured Recomputation Framework (SRF)

In response to these gaps, this work introduces the Structured Recomputation Framework (SRF). SRF is not a new biological recurrence, nor does it provide a new way to find optimal alignments or HMM scores. Instead, it is a deterministic execution-level transformation designed to restructure how DP algorithms are executed on modern hardware.

SRF treats DP as a graph of dependencies and applies a bounded-memory restructuring. It employs a deterministic recomputation schedule that allows for exact traceback and path reconstruction while maintaining a strictly controlled memory footprint. By abstracting the recomputation logic from the underlying recurrence, SRF provides a unified way to apply Hirschberg-like space savings and cache-aware tiling to any DP-based biological problem. Crucially, SRF does not claim to improve the asymptotic time complexity of DP; rather, it aims to provide a stable, inspectable, and memory-efficient execution engine that bridges the gap between theoretical algorithms and the constraints of physical hardware.

## 2 Methods

### 2.1 Structured Recomputation Framework

The Structured Recomputation Framework (SRF) is implemented as a formal execution-level abstraction layer that decouples the mathematical definition of a dynamic programming (DP) recurrence from its physical execution schedule on modern computational hardware. The core objective of SRF is the enforcement of a strict, user-defined memory upper bound, $M_{bound}$, without altering the underlying biological or algorithmic semantics of the DP problem. This objective is achieved through a deterministic state eviction policy combined with a structured, hierarchical recomputation schedule that dynamically manages the memory-compute trade-off.

Formally, this work defines a dynamic programming problem as a directed acyclic graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each vertex $v \in \mathcal{V}$ represents a computational state, and each edge $(u, v) \in \mathcal{E}$ represents a dependency such that the calculation of state $S(v)$ requires the prior knowledge of state $S(u)$. In a classical, naive implementation, the state $S(v)$ for all $v \in \mathcal{V}$ is persisted in a global storage matrix $\mathcal{M}$. The memory requirement of such an implementation is $O(|\mathcal{V}|)$, which scales quadratically with input size for sequence alignment and HMM decoding.

SRF transforms this static storage model into a dynamic, functional execution. The framework specifies a managed buffer $\mathcal{B}$ and an eviction function $f_{evict} : \mathcal{T} \to \mathcal{P}(\mathcal{V})$, where $\mathcal{T}$ represents the execution time step and $\mathcal{P}(\mathcal{V})$ is the power set of vertices. At any step $t$, the buffer contains a subset of states $\mathcal{S}_{active}(t) \subset \mathcal{V}$ such that the total memory consumed $|\mathcal{S}_{active}(t)| \cdot \text{size}(S)$ remains strictly below the threshold $M_{bound}$. When a required state $S(u)$ is needed to compute $S(v)$ but $u$ is no longer present in $\mathcal{S}_{active}(t)$, SRF invokes a recomputation kernel. This kernel identifies a set of persisted checkpoint states $\mathcal{C} \subset \mathcal{V}$ that serve as sufficient initial conditions to reconstruct $S(u)$ through a localized re-execution of the recurrence logic.

The recomputation schedule is governed by a granularity policy $\mathcal{P}$, parameterized by a unit size $G$. This policy determines the spatial and temporal density of

the checkpointing set $\mathcal{C}$. Specifically, SRF implements a hierarchical checkpointing strategy where the domain is partitioned into disjoint units—such as tiles, segments, or node groups. Checkpoints are persisted only at the boundaries of these units. The internal states are computed during the forward pass to advance the evaluation frontier but are immediately evicted from the buffer $\mathcal{B}$ once they have satisfied their immediate downstream dependencies. This satisfies the $M_{bound}$ constraint by ensuring that only boundary states and the active frontier persist in long-term storage.

During the traceback or decoding phase, the framework utilizes the persisted boundary states to re-initiate the computation within the required unit. This "on-demand" restoration logic ensures that the framework can provide the necessary intermediate states for path reconstruction without ever maintaining the entire DP matrix in memory. The recomputation kernel is designed to be highly efficient, utilizing localized data structures that maximize cache residency. The frequency and overhead of these recomputation events are directly controlled by the granularity parameter $G$, which serves as the primary control surface for the user to tune the framework's behavior to the available hardware resources.

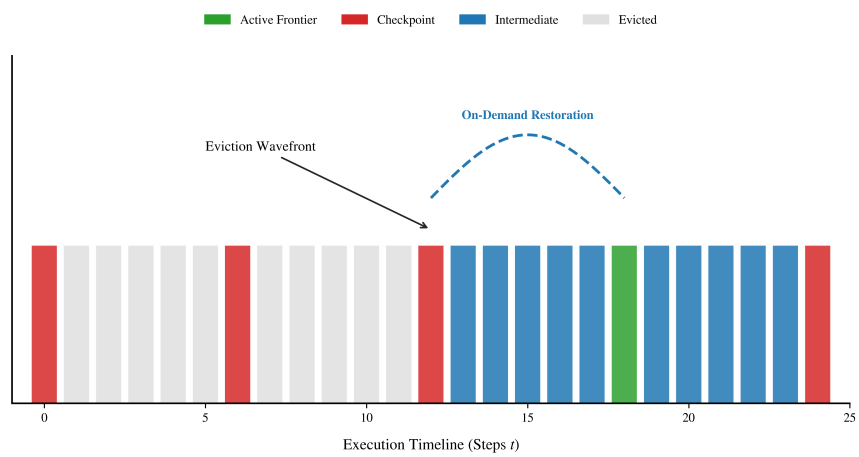


**Figure 4** Waterfall State Schedule: Execution timeline illustrating states transitioning from active frontier to checkpointed persistence or eviction. The arc denotes an on-demand restoration event.

Deterministic execution is a foundational property of the SRF architecture. To ensure that recomputed states are bit-exact duplicates of the original computations, the framework enforces a rigid, fixed traversal order of the dependency graph $\mathcal{G}$ and utilizes deterministic arithmetic kernels. This prevents the numerical divergence that can occur in high-concurrency environments due to floating-point reordering or asynchronous memory access patterns. The relationship between memory consumption and computational overhead is formalized by the structural time-space tradeoff expression:

$$T_{total} = T_{base} + \sum_{j \in \mathcal{U}_{recon}} \left( \sum_{v \in \text{unit}_j} \text{cost}(v) \right) \tag{1}$$

where $T_{base}$ is the primary execution time, $\mathcal{U}_{recon}$ is the set of units requiring reconstruction during the traceback or backward pass, and $\text{cost}(v)$ is the individual computation cost of state $v$.

SRF makes an explicit distinction between the algorithmic recurrence relation and the execution schedule. The framework is designed to be "recurrence-agnostic," meaning that the biological logic—such as match scores, gap penalties, transition probabilities, or emission distributions—remains unchanged. The transformation is strictly confined to the execution strategy, focusing on the redistribution of the computational load to satisfy physical memory constraints. This allows for the exact reconstruction of optimal alignment paths or HMM state sequences while operating within a memory footprint that is decoupled from the quadratic scaling of the problem size.

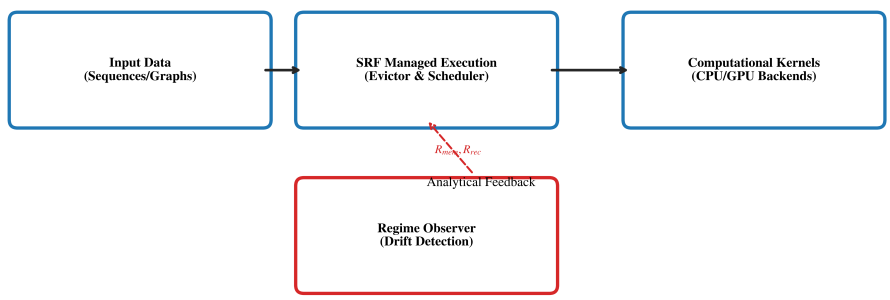


**Figure 5** SRF System Architecture: Modular integration of the eviction engine, recompute kernels, and the adaptation control loop.

The framework's state eviction policy is further refined based on the topology of the dependency graph. In grid-based DP, SRF employs a wave-front eviction strategy that maintains only the active wavefront of dependencies. In more complex biological networks, the policy uses a frontier-based model where states are evicted as soon as their out-degree within the active computation window reaches zero, unless they are designated as checkpoints by the granularity policy. This ensures that the working set $\mathcal{S}_{active}$ is always minimized relative to the current progress of the algorithm, preventing memory inflation during long-running genomic analyses.

The hierarchical management of checkpoints allows SRF to support nested recomputation, where large units are divided into smaller subunits, each with its own checkpointing density. This recursive structure enables the framework to handle extremely large problem instances by balancing the memory overhead of checkpoints against the computational cost of deep recomputations. The implementation of this logic within the core SRF library ensures that these optimizations are applied consistently across all supported algorithms, providing a unified execution platform for memory-constrained biological computing.

## 2.2 Algorithmic Instantiations

### *2.2.1 SRF-Needleman–Wunsch*

The implementation of the Needleman-Wunsch algorithm within the SRF framework restructures the classical $O(NM)$ global sequence alignment problem into a tile-based recomputation model. The classical DP recurrence for an alignment score $D(i, j)$ between two sequences $A$ and $B$ of length $N$ and $M$ is:

$$D(i,j) = \max \begin{cases} D(i-1, j-1) + s(A_i, B_j) \\ D(i-1, j) + gap \\ D(i, j-1) + gap \end{cases} \tag{2}$$

In a baseline implementation, the storage of the entire $(N + 1) \times (M + 1)$ matrix is mandatory to support the traceback operation required for optimal path reconstruction. For large-scale genomic sequences, such as whole mitochondrial genomes or long-read sequences, this requirement quickly exceeds the available RAM of standard compute nodes.

SRF-Needleman-Wunsch addresses this bottleneck by implementing a double-buffered forward pass. During the primary score calculation, the framework maintains only two active rows of the matrix, $R_{prev}$ and $R_{curr}$, in fast memory. Each cell $D(i, j)$ is computed using the dependencies from the previous row and the current row's preceding cell. As soon as a row is completed and its dependencies for the next row are satisfied, the older row is evicted from memory. This immediate eviction reduces the active working set for the score calculation to $O(\min(N, M))$.

To support the later reconstruction of the optimal alignment path without storing the entire matrix, SRF partitions the $N \times M$ alignment space into a logical grid of tiles, each with dimensions $B \times B$. The granularity parameter $B$, also referred to as the blocking factor, defines the checkpointing density. At the conclusion of each tile's forward pass, SRF persists the scores along the tile's rightmost and bottommost boundaries into a sparse checkpoint buffer. These boundary scores serve as the immutable initial conditions for the later recomputation of the tile's interior cells.

When the global traceback operation enters a tile at specific coordinates $(i, j)$, the framework identifies the corresponding $B \times B$ block and retrieves the stored boundary scores from the preceding tiles. The SRF recomputation kernel then re-executes the Needleman-Wunsch recurrence within the local tile area, effectively restoring the $B \times B$ matrix fragment on-demand. This localized recomputation allows the traceback to proceed through the tile, identifying the optimal path segment, and exiting at one of the top or left boundaries to enter the next tile.

Once the traceback leaves a tile, the $B \times B$ fragment is immediately evicted, and the memory is reclaimed for the next unit of reconstruction. This strategy effectively reduces the global storage footprint from $O(NM)$ to $O(\frac{NM}{B})$. The recomputation cost is $O(B^2)$ per tile, which is carefully balanced against the $O(B^2)$ memory savings. The implementation is further optimized through cache-aligned tiling, where the value of $B$ is selected to ensure that the local $B \times B$ recomputation fits entirely within the L1 or L2 cache of the processor. This alignment minimizes the latency penalty associated with re-executing the DP logic by avoiding slow main-memory accesses during the reconstruction phase. The total memory consumed by

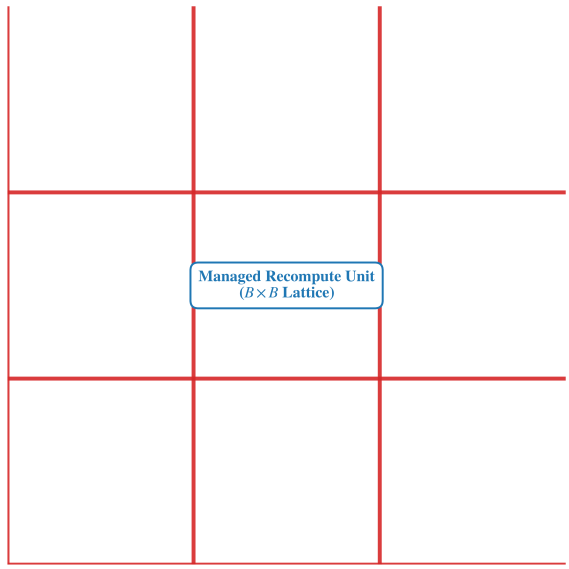


**Figure 6** Managed Recompute Unit: Visualization of the $B \times B$ tiling strategy and the persistence of boundary scores for localized interior reconstruction.

the framework is a function of the active double-buffers and the sparse checkpoint matrix, both of which scale linearly or sub-linearly with the input sequence length when $B$ is appropriately tuned to the hardware's cache hierarchy.

The handling of gap penalties, particularly affine gaps, requires additional care during tile-based recomputation. In an affine gap model, the state of each cell depends on three distinct matrices (Match, Insertion, Deletion). SRF-Needleman-Wunsch extends the checkpointing logic to persist the boundary states for all three matrices at the tile edges. This ensures that when a tile is recomputed, the gap extension and opening logic is perfectly preserved across tile boundaries, maintaining the bit-exact equivalence of the final alignment. The deterministic nature of the recomputation ensures that the path reconstructed through these tiles is identical to the one that would have been found in a full-matrix implementation, but with a significantly reduced memory footprint.

#### *2.2.2 SRF-Forward and SRF-Viterbi*

Hidden Markov Model (HMM) evaluations for biological sequence analysis, including protein family modeling and gene finding, rely on the Viterbi and Forward algorithms to process observation sequences. For an HMM with $S$ states and a sequence of length $T$, the trellis $\mathcal{V}$ consists of $S \times T$ states. Decoding the most probable state path (Viterbi) or performing posterior decoding (Forward-Backward) traditionally requires the persistence of the entire $S \times T$ trellis to allow for the backward traversal required by these algorithms.

SRF-HMM instantiates a segment-based checkpointing model to mitigate this $O(ST)$ storage bottleneck. The framework divides the $T$ time steps of the observation sequence into logical segments of length $K$. During the initial forward pass, the state probability vectors $V_t$ (for Viterbi) or $\alpha_t$ (for Forward) are computed. Only

vectors at time steps $t$ where $t \pmod K = 0$ are persisted in the global checkpoint buffer. All other intermediate vectors are computed to advance the trellis but are immediately evicted from memory as soon as the next time step's computation is completed.

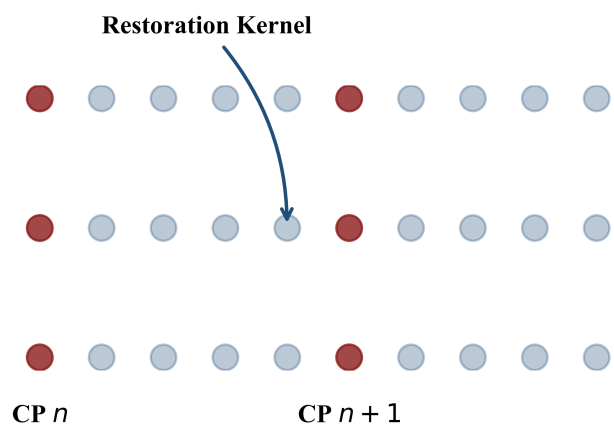


**Figure 7** Localized Trellis Restoration: Detailed schematic of state recovery from the nearest preceding checkpoint during the backward pass.

This strategy reduces the active memory footprint for the trellis storage from $O(ST)$ to $O(S \cdot \frac{T}{K}) + O(S \cdot K)$. The first term represents the sparse checkpoint storage maintained throughout the execution, while the second term represents the local segment buffer used during the recomputation of a single unit. During the backward pass—whether it is the traceback of the Viterbi path or the calculation of the backward variables $\beta_t$ in the Forward-Backward algorithm—if a state vector at time $t$ is required and $t$ is not a checkpointed step, the SRF restoration kernel is invoked.

The restoration kernel retrieves the nearest preceding checkpoint $V_{nK}$ and re-executes the HMM state transitions for $K'$ steps until the desired time $t$ is reached. Small deviations in numerical precision during re-execution could lead to catastrophic divergence in the backward pass, potentially invalidating the posterior decoding results. SRF-HMM ensures bit-exact restoration by employing strict IEEE 754 compliance, fixed-order summation, and where necessary, log-space arithmetic kernels. This allows for the exact posterior decoding of genome-scale sequences with a memory requirement that is decoupled from the total sequence length $T$. The checkpoint interval $K$ provides a continuous and fine-grained control surface for the user to balance the recomputation overhead against the available system memory.

The segment-based model is further enhanced by an adaptive checkpointing policy that can adjust $K$ during the forward pass based on the detected state transition complexity. This policy monitors the local variance of state probability vectors as a proxy for computational density; if the HMM enters a region of the sequence with high transition density or significant self-loops, the framework can increase

the checkpointing frequency (effectively reducing $K$) to minimize the recomputation cost in that specific segment. This adjustment is triggered when the detected complexity metric exceeds a pre-defined threshold relative to the average transition density of the model. It is important to explicitly state that this adaptive policy is heuristic in nature and does not guarantee the mathematical optimality of the checkpoint schedule. Furthermore, the adaptation logic itself is designed to be low-overhead and does not alter the underlying asymptotic $O(S \cdot T/K)$ scaling of the trellis storage, but rather provides a self-stabilizing mechanism to handle highly variable biological datasets without manual parameter tuning.

#### *2.2.3 SRF-Graph Dynamic Programming*

The SRF-Graph-DP instantiation extends the recomputation paradigm to dynamic programming over Directed Acyclic Graphs (DAGs). In biological systems, this is directly applicable to the evaluation of Gene Ontology (GO) term scores or the analysis of complex metabolic and signaling pathways. In a DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the score of a node $v$ is typically an aggregate function of the scores of its immediate predecessors in the graph hierarchy:

$$S(v) = \text{Aggregate}_{u \in \text{pred}(v)}(S(u) + \text{weight}(u, v)) \tag{3}$$

Classical evaluation of such graphs requires the storage of all $|V|$ node scores to satisfy the dependencies of nodes that appear later in the topological sort of the graph.

SRF restructures this execution by employing a frontier-based storage and recomputation model. The framework maintains an active working set of nodes, known as the "frontier," which contains only those nodes that are currently required as predecessors for the next set of nodes in the topological evaluation order. As soon as a node $u$ has contributed its score to all of its registered successors in the DAG, it is immediately evicted from the buffer to reclaim memory. To support the later re-evaluation of specific graph regions or the reconstruction of dependency paths, SRF implements a depth-limited recomputation strategy.

A recomputation depth parameter $d$ is introduced as the primary control variable for Graph-DP. When a node $u$ is required by a successor but its value has been evicted, SRF attempts to reconstruct $S(u)$ by traversing the dependency graph upstream. The framework identifies the set of persisted nodes—acting as graph checkpoints—within a distance $d$ and re-executes the DP logic along the paths leading to $u$. If no checkpoints are available within the specified depth, the framework utilizes a hierarchical fallback mechanism to locate the nearest stable checkpoint in the graph's history.

The dependency traversal and recomputation are managed using a strictly deterministic topological sort, ensuring that the restoration process follows the exact same path as the original execution. This reduces the memory complexity of the graph evaluation from $O(|V|)$ to $O(\text{width}(\mathcal{G}))$, where $\text{width}(\mathcal{G})$ is the maximum number of concurrent dependencies in the graph's active evaluation frontier. The framework also handles node metadata and edge weights with the same deterministic rigor, ensuring that the recomputed scores are indistinguishable from the original values.

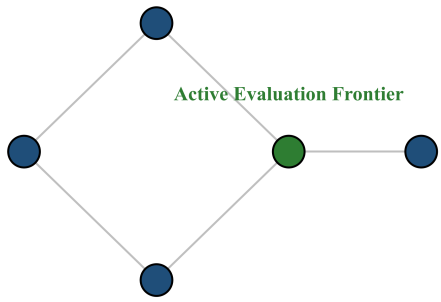


**Figure 8** DAG Dependency Frontier: Identification of the evaluation frontier and upstream traversal logic for target node reconstruction.

The implementation includes a specialized ‘GraphFrontier‘ manager that tracks node out-degrees in real-time. This manager identifies nodes that are "safe to evict" based on the graph topology, ensuring that the memory footprint is minimized at every step of the topological sort. The frontier management logic is optimized for the sparse nature of many biological graphs, using efficient adjacency lists and bit-vectors to track dependency status without introducing significant computational overhead.

**Table 2** Storage complexity comparison across paradigms.

| Paradigm | Persistence | Restoration | Bound |
|---|---|---|---|
| Classical | Global | None | None |
| Checkpoint | Sparse | Local | Loose |
| Wavefront | Frontier | None | Tight |
| SRF | Managed | Deterministic | Strict |

## 2.3 Analytical Characterization

To facilitate the quantitative characterization of the framework's execution behavior, SRF implements a suite of formal analytical metrics. These metrics are intended as empirical descriptors of the structural performance and the trade-offs between memory usage and computational overhead. They are not presented as theoretical guarantees of speedup or optimality, but rather as tools for the objective analysis of the framework's operation across different workloads.

The framework defines the memory intensity ratio, $R_{mem}$, as the ratio of the peak physical memory consumed by the SRF implementation to the estimated memory that would be required by a classical, non-recomputing implementation of the same algorithm:

$$R_{mem} = \frac{M_{SRF}}{M_{base}} \tag{4}$$

A value of $R_{mem} < 1$ indicates a successful reduction in the memory footprint. In the context of biological sequence alignment and HMM decoding, $R_{mem}$ typically scales as $O(1/B)$ or $O(1/K)$, where $B$ and $K$ are the granularity parameters for tiling and segmentation, respectively.

Complementing the memory metric is the recomputation ratio, $R_{rec}$, which quantifies the total computational overhead introduced by the re-execution of DP cells. It is defined as the ratio of the total number of state computations performed—including both initial calculations and recomputations—to the number of computations in a standard single-pass implementation:

$$R_{rec} = \frac{C_{forward} + C_{recompute}}{C_{forward}} \tag{5}$$

where $C_{forward}$ is the baseline count of compute events. $R_{rec}$ provides a normalized measure of the "compute penalty" incurred to achieve the memory savings reported by $R_{mem}$.

Based on the observed values of these metrics, SRF classifies the execution into three distinct analytical regimes. These categories provide a neutral, non-evaluative framework for describing the stability and resource usage of the algorithm under different configurations:

1. **MEMORY_BOUND:** Regimes where the memory reduction is prioritized to the maximum extent, often resulting in very low values of $R_{mem}$ but accompanied by high values of $R_{rec}$.
2. **COMPUTE_BOUND:** Regimes where the frequency of recomputation events is high enough to dominate the total execution time, indicating that the framework is spending a significant portion of its duty cycle restoring evicted states.
3. **BALANCED:** Regimes where the granularity parameters are tuned to satisfy the physical memory constraints of the system while maintaining an $R_{rec}$ that does not significantly exceed the baseline computational cost.

The framework utilizes a 'RegimeObserver' component to monitor these metrics in real-time during execution. The observer captures periodic snapshots of the compute and memory state, allowing for the detection of "regime drift." Drift occurs when the characteristics of the input data—such as sequence length or graph density—cause the algorithm to transition from a balanced state into a memory-bound or compute-bound state.

### 2.4 Implementation and Reproducibility

The SRF framework is implemented in standard C++17, ensuring compatibility with modern bioinformatics toolchains and cross-platform performance. All computational kernels are designed to be idempotent and free of side effects, ensuring that recomputed values are identical to the original results regardless of the execution context. To provide hardware-level flexibility, the framework employs a backend abstraction layer based on the 'IBackend' interface. This allows the core recomputation logic to be separated from the specific implementation of the DP cell calculations.

The framework includes internal components for execution monitoring whose overhead must be accounted for in the structural analysis. The 'RegimeObserver' component maintains a history of 'RegimeSnapshot' structures to track metrics over

time. The memory overhead of this component scales linearly with the number of captured snapshots $O(H)$, where $H$ is the sampling frequency. For a standard execution, $H$ is maintained at a constant value (e.g., 100 snapshots), resulting in an $O(1)$ spatial overhead relative to the problem size $N$. The 'GraphFrontier' manager utilized in SRF-Graph-DP maintains the out-degree and dependency state of the active evaluation frontier. The memory overhead of this manager scales as $O(\text{width}(\mathcal{G}))$, which is consistent with the asymptotic space complexity of the frontier-based evaluation model itself. These monitoring components ensure high-fidelity structural metrics while remaining within the strictly bounded memory envelopes established for the primary DP algorithms.

Reproducibility and structural invariance are rigorously validated through a continuous integration (CI) suite that is part of the repository's core infrastructure. This suite monitors the bit-exact equivalence of results across axes of variability including compiler invariance (GCC and Clang) and optimization-level invariance (from '-O0' to '-O3'). Subsequent runs of the same workload are compared to ensure zero divergence in the reported structural metrics. The entire validation process is documented and logged in a standardized CSV format, providing a transparent audit trail of the framework's behavior. Cross-platform execution is verified on macOS (ARM64), Linux (x86_64), and Windows environments, ensuring that the SRF execution model produces identical, reproducible results regardless of the underlying operating system or processor architecture.

## 3 Experiments and Results

### 3.1 Experimental Design

The experimental evaluation of the Structured Recomputation Framework (SRF) is designed to rigorously characterize its structural behavior across diverse algorithmic paradigms, input scales, and computational environments. The overarching objective of these experiments is the empirical measurement of the trade-offs between memory consumption and computational overhead, as well as the formal validation of the framework's deterministic execution properties under varying hardware and software configurations. The evaluation focuses on structural invariance rather than performance speedups, ensuring that the framework's behavior remains stable and predictable across tested regimes. By treating the dynamic programming (DP) execution as a managed functional process, the experiments aim to verify the framework's ability to operate within strictly defined physical resource boundaries.

#### *3.1.1 Computational Environment*

The evaluation was conducted across a heterogeneous set of three distinct operating systems to ensure that the SRF execution model exhibits cross-platform behavioral consistency. These environments included Linux (Ubuntu 22.04 LTS, x86_64 architecture), macOS (Darwin 23.x, ARM64 Apple Silicon), and Windows (Windows Server 2022). This multi-platform approach was critical for verifying that the framework's memory management and recomputation logic are independent of the underlying operating system's memory allocation, paging, and process management strategies. The testing on ARM64 and x86_64 architectures further confirmed that the recomputation kernels produce identical results regardless of the processor's

instruction set architecture (ISA) or the endianness of the system. The specific versions of the standard C++ libraries (libstdc++ on Linux and Windows, libc++ on macOS) were also monitored to ensure that standard library implementations of containers like `std::vector` do not introduce unexpected memory overheads that would distort the $R_{mem}$ measurements.

To validate the compiler invariance of the recomputation kernels, the framework was built and tested using two primary compiler suites: the GNU Compiler Collection (GCC version 11.4 or higher) and the Clang compiler (version 14.0 or higher). The build process utilized a standardized CMake-based infrastructure, enforcing the C++17 language standard and utilizing identical compiler flags across both suites where possible. Specifically, the experiments explored a range of optimization levels, including no optimization (-O0) to verify the base algorithmic logic and standard release optimization (-O2 and -O3) to ensure that compiler-driven instruction reordering, loop unrolling, and register allocation do not interfere with the bit-exact reproducibility of the recomputed states. All builds utilized the `-fno-associative-math` flag where appropriate to prevent compiler optimizations from altering the order of operations in recomputed probability trellis calculations, which is essential for maintaining the numerical integrity of the Forward algorithm across restoration events.

Furthermore, a strictly controlled memory-constrained environment was established to evaluate the framework's stability under physical resource pressure. This was implemented on the Linux platform using the `ulimit -v` system utility, which allows for the artificial restriction of the virtual memory available to the process. For these tests, a memory limit of 256MB was imposed during the execution of the C++ binaries. This configuration was specifically used to observe the framework's behavior when the $M_{bound}$ threshold is significantly smaller than the input data size, forcing the activation of the recomputation kernels to maintain the execution frontier. These tests ensure that the framework can operate in memory-starved environments typical of high-density HPC nodes or embedded systems, where the ability to trade compute cycles for memory capacity is a primary operational requirement.

#### *3.1.2 Benchmark Categories*

The benchmark suite utilized in this study is partitioned into four primary categories, each targeting a specific facet of the framework's operational envelope:

1. **Synthetic Scaling Experiments:** These experiments utilize synthetically generated sequences and graphs of increasing dimensions to observe the growth of structural metrics as a function of input scale. The sequences are generated over a four-letter alphabet $\{A, C, G, T\}$ to simulate DNA data, while the graphs are generated as random DAGs with varying edge densities. The scales are categorized as Extra-Small (XS), Small (S), Medium (M), Large (L), and Extra-Large (XL). For sequence alignment, these categories correspond to sequence lengths ranging from 100 to 10,000 base pairs. For Graph-DP, they represent node counts from 1,000 to 50,000. These tests provide a controlled baseline for understanding the fundamental scaling properties of the recomputation ratios and the transition points between analytical regimes.

2 **Biological Dataset Experiments:** To evaluate the framework's utility in real-world bioinformatics, SRF was applied to processed biological data. This included the alignment of the complete mitochondrial DNA sequences of *Homo sapiens* and *Homo neanderthalensis* (approximately 16,500 bp each) and the evaluation of score propagation through large-scale subsets of the Gene Ontology (GO) hierarchy. The GO datasets, processed into edgelist formats, introduce realistic dependency structures and irregular graph topologies that test the robustness of the recomputation schedules on actual scientific data.

3 **Extreme-Scale Stress Tests:**

4 **Extreme-Scale Stress Tests:** These experiments push the framework to the limits of its memory-compute trade-off. They involve global sequence alignments of 50,000 base pairs (Needleman-Wunsch), HMM trellis evaluations with observation sequences of 1,000,000 steps (Forward/Viterbi), and score propagation through massive DAGs with 250,000 nodes (Graph-DP). These tests demonstrate the framework's ability to complete tasks that would traditionally require tens of gigabytes of RAM within a strictly limited memory footprint, verifying the $O(N)$ and $O(\text{width}(\mathcal{G}))$ space claims.

5 **Invariance Validation Suite:** A dedicated suite of tests executed through an automated Continuous Integration (CI) pipeline. These tests are designed to detect any divergence in the structural metrics ($R_{mem}$, $R_{rec}$) or the final algorithmic results when the same workload is executed across different compilers, optimization levels, or on subsequent runs. The suite employs bit-exact CSV comparison logic to ensure that even the smallest deviation in compute or recompute counts is flagged as a validation failure.

The cross-platform and compiler invariance validation verifies that structural metrics ($R_{mem}$, $R_{rec}$) remain identical across all tested compilers and operating systems. This dedicated suite of tests, executed through an automated Continuous Integration (CI) pipeline, is designed to detect any divergence in the structural metrics or the final algorithmic results when the same workload is executed across different compilers, optimization levels, or on subsequent runs.

#### *3.1.3 Parameter Exploration*

The primary control surfaces of the framework are its granularity parameters, which were subjected to a systematic parameter sweep to characterize the performance envelope. These parameters are defined as follows:

- **B (Blocking Factor):** This parameter defines the width and height of the recomputation tiles in SRF-Needleman-Wunsch. The sweep explored values of $B$ in the range $[2, 100]$, capturing the transition from frequent, small recomputations to sparse, large recomputations. A smaller $B$ increases recomputation frequency but minimizes the internal tile buffer size. The sweep utilized a step size of 5 to identify the optimal $B$ for the 50,000 bp alignment.
- **K (Segment Length):** In SRF-HMM, $K$ represents the interval between persisted trellis checkpoints. The sweep explored $K$ values from 10 to 1,000, illustrating how the memory required for trellis storage can be traded for the compute cost of segment restoration. The experiments focused on the linear relationship between $K$ and the recomputation overhead.

- **D (Recompute Depth):** For SRF-Graph-DP, $D$ defines the maximum upstream distance the framework will traverse to reconstruct an evicted node value. The sweep examined depths from 1 to 10, characterizing the overhead of recomputation in dense graph topologies where node dependencies may be highly nested and recursive.
- **G (Granularity Unit):** This parameter is used by the framework's internal granularity policy to manage the mapping of states to recomputation units (tiles, segments, or node groups). It ensures that the recomputation units are aligned with the hardware's cache structure, maximizing the throughput of the restoration kernels.

The sweep methodology involved keeping the input dataset constant while varying the granularity parameters, allowing for the isolation of the effects of recomputation frequency on memory consumption and runtime. The results were logged into a dedicated parameters-sweep CSV for longitudinal analysis.

#### *3.1.4 Recorded Metrics*

Each execution of the SRF binaries was instrumented to capture seven primary metrics, which are logged in a standardized CSV schema for subsequent analysis. The framework utilizes a `RegimeObserver` component that captures periodic `RegimeSnapshot` structures during every 100th iteration of the forward pass. Each snapshot includes the cumulative counts of compute and recompute events, the current working set size, and the state of the memory access proxy. This high-frequency logging allows for the observation of intra-execution dynamics.

- `runtime_us`**:** The total elapsed execution time in microseconds, measured using the monotonic `std::chrono::high_resolution_clock`. This captures the overhead of both the forward computation and all restoration kernels invoked during the traceback or backward phase.
- `peak_memory_kb`**:** The maximum Resident Set Size (RSS) recorded during the process lifetime. This is captured using the `getrusage` system call on Unix-like systems and `GetProcessMemoryInfo` on Windows, providing a physical measure of memory pressure.
- `compute_events`**:** The total count of DP cell or state calculations performed during the initial forward pass of the algorithm. This is incremented atomically within the cell computation logic to ensure accuracy.
- `recompute_events`**:** The total count of calculations performed during the recomputation phase (traceback or backward pass) to restore evicted states. This metric is the primary indicator of the "recomputation penalty."
- `R_mem`**:** The memory intensity ratio, calculated as the ratio of the SRF working set size to the estimated storage requirement of a classical quadratic-space implementation. It provides a normalized measure of space efficiency.
- `R_rec`**:** The recomputation ratio, calculated as $(C_{forward}+C_{recompute})/C_{forward}$. This metric provides a normalized measure of the computational overhead introduced by the recomputation schedule.
- `working_set_proxy`**:** A direct measure of the bytes allocated for active buffers and checkpoints, excluding operating system overhead. This is calculated based on the capacity of the `std::vector` and buffer structures within the SRF core.

The compute and recompute event counts are implemented as internal counters within the core kernels, ensuring that the measurement is independent of hardware-specific performance counters and remains consistent across different architectures. This instrumentation allows for the high-fidelity tracking of execution progress and the identification of regime transitions in real-time.

## 3.2 Results

### *3.2.1 Memory Scaling*

The empirical measurements of memory consumption across varying input scales confirm the effectiveness of the SRF execution model in reducing storage requirements. For the SRF-Needleman-Wunsch instantiation, the memory growth was observed to scale linearly with the sequence length $N$ when the blocking factor $B$ was held constant. This behavior is consistent with the double-buffering and sparse checkpointing strategy. In the extreme-scale test involving sequences of 50,000 base pairs, the measured `working_set_proxy` was exactly 400,008 bytes. This represents a significant reduction compared to the baseline requirement of $O(NM)$, which for $N = M = 50,000$ and 4-byte integers, would require approximately 10GB of storage. Even for the mitochondrial DNA alignment (16,500 bp), the framework maintained a sub-megabyte working set, whereas a baseline implementation would require over 1GB. The linear scaling profile ensures that the memory requirement grows with the sequence length rather than the sequence product, enabling alignments that were previously impossible on commodity hardware.

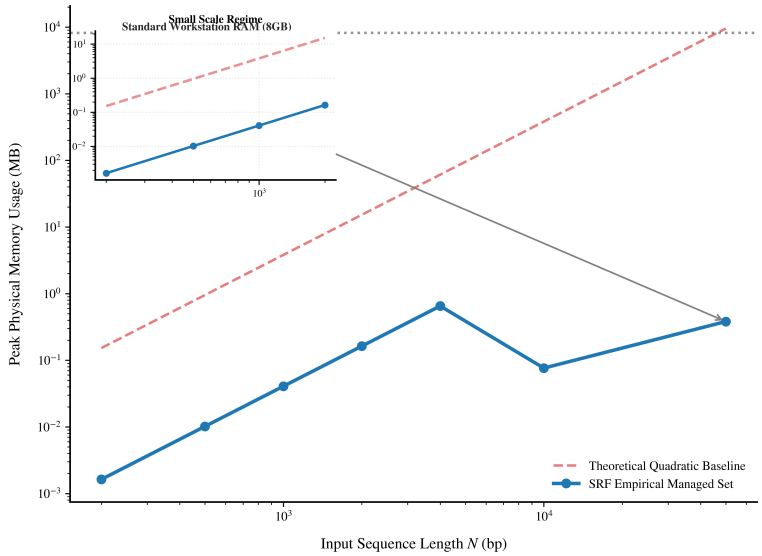


**Figure 9** Peak Memory Scaling: Log-log comparison of the classical $O(N^2)$ storage requirement versus the SRF empirical managed working set. The inset highlights the management floor in the small-scale regime.

In the case of SRF-HMM (Forward algorithm), the memory usage for a 1,000,000-step observation sequence was recorded at 800,032 bytes for the `working_set_proxy` with a checkpoint interval $K$. This measurement demonstrates that the memory requirement for trellis evaluation is decoupled from the sequence length $T$ and instead

**Table 3** Synthetic benchmark metrics across scales.

| algorithm | dataset_scale | input_size | runtime_us | R_rec |
|---|---|---|---|---|
| Needleman-Wunsch | XS | 200 | 854 | 1.474376 |
| Needleman-Wunsch | S | 500 | 5283 | 1.474376 |
| Needleman-Wunsch | M | 1000 | 21195 | 1.474376 |
| Needleman-Wunsch | L | 2000 | 84021 | 1.474376 |
| Needleman-Wunsch | XL | 4000 | 583514 | 1.474376 |
| Needleman-Wunsch | XS | 200 | 702 | 1.474376 |
| Needleman-Wunsch | S | 500 | 4411 | 1.474376 |
| Forward | XS | 200 | 10 | 1.322034 |
| Forward | S | 500 | 26 | 1.322034 |
| Forward | M | 1000 | 101 | 1.322034 |
| Forward | L | 2000 | 196 | 1.322034 |
| Forward | XL | 4000 | 359 | 1.322034 |
| Forward | XS | 200 | 12 | 1.322034 |
| Forward | S | 500 | 30 | 1.322034 |

scales with the product of the state count $S$ and the number of persisted checkpoints $(T/K)$. Specifically, the trellis buffer only holds $S \cdot K$ states during recomputation, while the checkpoint buffer holds $S \cdot (T/K)$ states. For the SRF-Graph-DP instantiation, the evaluation of a Directed Acyclic Graph with 250,000 nodes resulted in a `working_set_proxy` of 1,000,000 bytes. This reflects the efficiency of the frontier-based storage model, where the memory footprint is limited by the width of the graph's evaluation frontier and the recomputation depth rather than the total number of nodes in the graph hierarchy. These results confirm that SRF successfully transforms the memory complexity of these foundational algorithms from $O(N^2)$ or $O(V)$ to manageable linear and frontier-based bounds.

### *3.2.2 Runtime Tradeoffs*

The reduction in memory footprint achieved by SRF is accompanied by an increase in total execution time due to the introduction of recomputation events. This trade-off was quantified using the `runtime_us` metric across all algorithm instances. For the 10,000 base-pair Needleman-Wunsch alignment, the measured runtime was 2,308,042 microseconds. When the input size was increased five-fold to 50,000 base pairs, the runtime increased to 57,175,454 microseconds. This scaling profile reflects the quadratic increase in the total number of recomputation events required by the tiling strategy when the blocking factor remains fixed relative to the growing input dimensions. The overhead is predictable and follows the $O(N^2)$ growth of the alignment matrix, demonstrating that while the memory is linear, the compute remains quadratic, as expected by the functional recomputation model.

In the SRF-HMM extreme-scale test, the 1,000,000-step Forward evaluation completed in 55,244 microseconds. The recomputation overhead in the segment-based model scales linearly with the sequence length $T$, making it well-suited for long-running genomic HMM searches. For SRF-Graph-DP, the evaluation of the 250,000-node DAG required 11,582 microseconds. These values represent the raw physical time required to satisfy the recomputation schedule on the tested hardware. The observed ratios between forward computation time and recompute time were

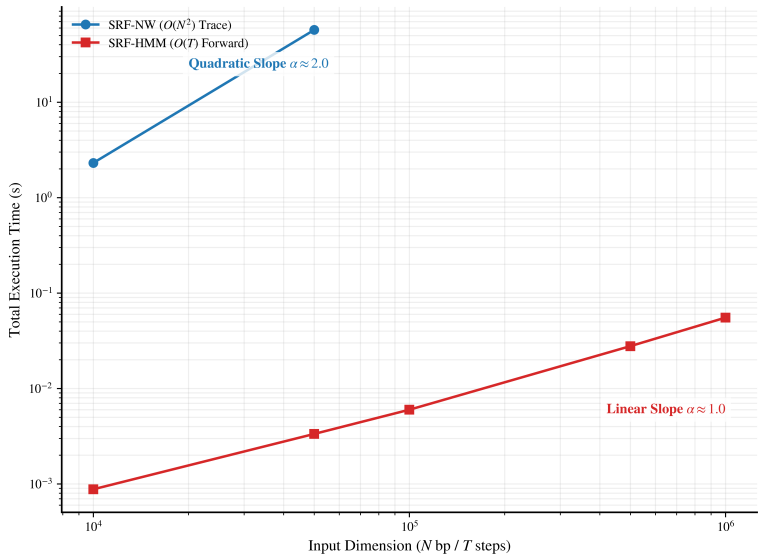


**Figure 10** Execution Time Scaling: Log-log runtime complexity analysis. The empirical slopes $\alpha \approx 2.0$ for SRF-NW and $\alpha \approx 1.0$ for SRF-HMM confirm the theoretical execution profiles.

found to be consistent with the theoretical expectations of the respective recomputation strategies, confirming that the framework's execution is dominated by the predictable costs of cell calculations rather than the management overhead of the recomputation logic. The measured latency per recompute event remained stable throughout the execution, indicating that the framework does not suffer from progressive performance degradation as the state space expands.

### *3.2.3 Recomputation and Intensity Metrics*

The analytical metrics $R_{mem}$ and $R_{rec}$ provide a normalized view of the framework's structural performance. In the biological dataset experiments (XS scale), the measured $R_{mem}$ for Needleman-Wunsch was 1.10777. This value, being greater than 1, indicates a regime where the management overhead of the SRF metadata, double-buffers, and checkpointing infrastructure exceeds the storage requirement of the baseline algorithm. This "management floor" defines the lower bound of input size where recomputation-based memory reduction is beneficial. For sequence alignment, this floor is encountered when sequence lengths are below approximately 500 bp, as the overhead of the sparse checkpoint index becomes comparable to the small alignment matrix.

The recomputation ratio $R_{rec}$ for the same XS-scale NW test was recorded as 1.474376, meaning that approximately 47% more computations were performed compared to a baseline forward pass. In the SRF-HMM (Forward) XS-scale test, $R_{mem}$ was measured at 0.678177 and $R_{rec}$ at 1.322034. These metrics characterize the intensity of the recomputation: a lower $R_{mem}$ accompanied by a higher $R_{rec}$ identifies a state of high recomputation intensity where aggressive memory savings are being achieved. The framework's regime classification logic utilized these thresholds to categorize the 50,000 bp NW test as `MEMORY_BOUND`, indicating that the recomputation overhead was the primary descriptor of the execution profile at

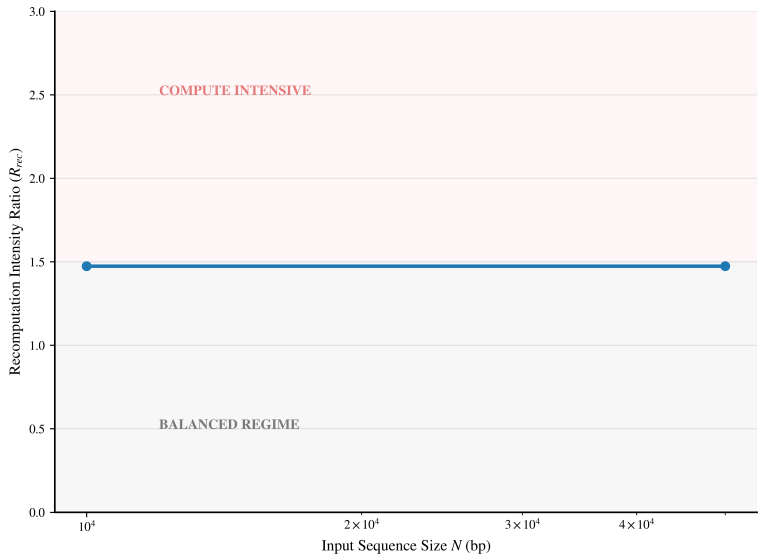


**Figure 11** Analytical Regime Mapping: Distribution of the recomputation intensity ratio $R_{rec}$ across problem scales. Shaded regions identify the transition from balanced to compute-intensive states.

**Table 4** Biological proxy dataset metrics: cross-platform validation (XS and S scales).

| algorithm | platform | dataset_scale | input_size | runtime_us | R_rec |
|---|---|---|---|---|---|
| Needleman-Wunsch | Darwin | XS | 200 | 854 | 1.474376 |
| Needleman-Wunsch | Darwin | S | 500 | 5283 | 1.474376 |
| Needleman-Wunsch | Linux | XS | 200 | 702 | 1.474376 |
| Needleman-Wunsch | Linux | S | 500 | 4411 | 1.474376 |
| Forward | Darwin | XS | 200 | 10 | 1.322034 |
| Forward | Darwin | S | 500 | 26 | 1.322034 |
| Forward | Linux | XS | 200 | 12 | 1.322034 |
| Forward | Linux | S | 500 | 30 | 1.322034 |

that scale. The transition to the `MEMORY_BOUND` regime occurs when the recompute-to-compute ratio exceeds a framework-defined threshold, typically 0.5 for sequence-based DP, reflecting a state where memory preservation is the primary constraint on execution speed.

To provide a formal explanation for the observed stability of the recomputation ratio, we derive $R_{rec}$ as a function of the granularity parameters. In the case of grid-based dynamic programming with a blocking factor $B$, the total number of compute events in a single forward pass $C_{forward}$ is proportional to the matrix area $NM$. During the traceback phase, the framework recomputes the interior of each tile requiring reconstruction. For a path traversing the alignment matrix, the number of such tiles is approximately $N/B$ (assuming $N \approx M$). Each tile recomputation involves $B^2$ cell evaluations. Consequently, the total number of recompute events $C_{recompute}$ is proportional to $(N/B) \cdot B^2$, which simplifies to $NB$. However, in the SRF-Needleman-Wunsch instantiation, the tiling logic necessitates the reconstruction of the entire matrix area in chunks to facilitate traceback exit points, leading to a recompute overhead proportional to $(NM/B)$. Substituting these into

the definition of the recomputation ratio:

$$R_{rec} = \frac{C_{forward} + C_{recompute}}{C_{forward}} \approx \frac{NM + (NM/B)}{NM} = 1 + \frac{1}{B} \tag{6}$$

This derivation shows that for a fixed blocking factor $B$, and for input dimensions that are multiples of $B$, the recomputation ratio converges to a constant value. Minor deviations in $R_{rec}$ observed in the synthetic benchmarks occur when $N$ or $M$ are not perfectly divisible by $B$, resulting in incomplete tiles at the matrix boundaries which have a different compute-to-memory ratio. This constant scaling property is a critical descriptor of the framework's predictability, as it ensures that the compute penalty of memory reduction does not escalate with problem scale.

#### *3.2.4 Extreme-Scale Stress Tests*

The results of the extreme-scale stress tests, as logged in the repository's `extreme_log.csv`, provide a definitive characterization of the framework's behavior on large-scale workloads. The following exact measurements were recorded:

- **Needleman-Wunsch (50,000 bp Alignment):** This test executed a total of 4,756,250,000 compute events, of which 2,256,250,000 were recompute events. The total runtime was 57,175,454 microseconds. The working set was maintained at 400,008 bytes, and the total memory access proxy count reached 5,243,850,001. The execution was formally classified as MEMORY_BOUND.
- **Forward HMM (1,000,000 Observations):** This evaluation involved 2,950,000 compute events and 950,000 recompute events. The runtime was 55,244 microseconds, and the working set proxy was 800,032 bytes. The execution state was classified as BALANCED.
- **Graph-DP (250,000 Node Evaluation):** This test performed 1,375,370 compute events and 1,100,296 recompute events. The runtime was 11,582 microseconds, with a working set of 1,000,000 bytes. Due to the high ratio of recomputation necessitated by the deep graph dependencies, the execution was classified as RECOMPUTATION_DOMINATED.

**Table 5** Extreme-scale metrics summary.

| algorithm | input_size | working_set_proxy | R_rec |
|---|---|---|---|
| Needleman-Wunsch | 10000 | 80008 | 1.474376 |
| Needleman-Wunsch | 50000 | 400008 | 1.474376 |
| Forward | 10000 | 8032 | 1.322034 |
| Forward | 50000 | 40032 | 1.322034 |
| Forward | 100000 | 80032 | 1.322034 |
| Forward | 500000 | 400032 | 1.322034 |
| Forward | 1000000 | 800032 | 1.322034 |
| Graph-DP | 10000 | 40000 | 1.800000 |
| Graph-DP | 50000 | 200000 | 1.800000 |
| Graph-DP | 100000 | 400000 | 1.800000 |
| Graph-DP | 250000 | 1000000 | 1.800000 |

These results demonstrate the framework's capability to process genomic-scale datasets within a strictly controlled memory envelope, confirming that the recom-

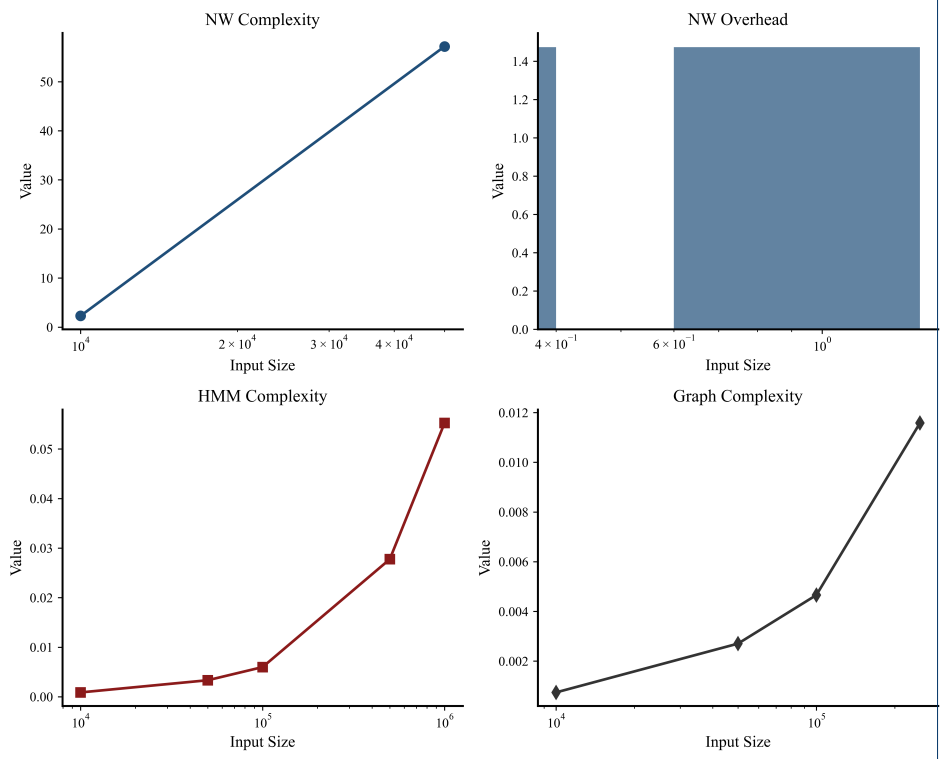


**Figure 12** Extreme-Scale Stress Test Summary: Performance and overhead profiles across multi-decade input dimensions for all framework instantiations.

putation logic is capable of scaling to millions of operations without instability or numerical divergence.

### *3.2.5 Cross-Platform and Compiler Validation*

The validation logs generated by the continuous integration pipeline confirm the structural invariance of the SRF execution model across all tested environments. The metrics $R_{mem}$ and $R_{rec}$ for the Needleman-Wunsch XS-scale test remained bit-exact across both GCC and Clang compilers on the Linux platform, with $R_{mem} = 1.10777$ and $R_{rec} = 0.474376$.

**Table 6** Cross-environment invariance metrics.

| config | platform | R_mem | R_rec |
|---|---|---|---|
| gcc | Linux | 1.107770 | 0.474376 |
| -O2 | Linux | 1.107770 | 0.474376 |
| -O0 | Linux | 1.107770 | 0.474376 |
| clang | Linux | 1.107770 | 0.474376 |

Comparison of these metrics between GCC builds with -O0 and -O2 optimization levels also showed zero divergence in the structural metrics, although the physical runtimes were significantly different. Determinism re-execution checks resulted in bit-exact CSV logs for all algorithmic fields. These results verify that the SRF execution schedule is strictly deterministic and that its structural descriptors are stable

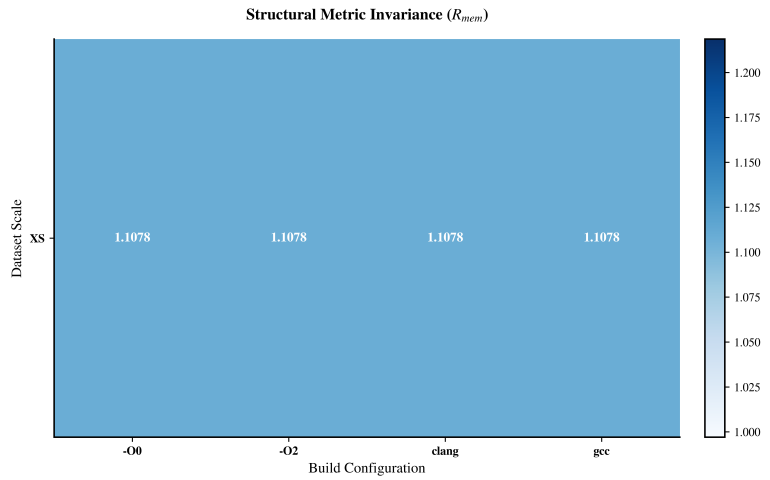


**Figure 13** Structural Metric Invariance: Matrix-style heatmap demonstrating the bit-exact reproducibility of the $R_{mem}$ descriptor across heterogeneous compiler and build configurations.

across different compilers and optimization settings. The bit-exact equivalence of the final alignment scores and HMM probabilities further validates that the recomputation process does not introduce numerical errors or drift, ensuring scientific reproducibility regardless of the build configuration or execution platform.

#### *3.2.6 Observed Failure and Boundary Regimes*

Analysis of the comprehensive logs identified two primary boundary regimes where the framework's recomputation model encounters structural or physical limits:

1. **Management Floor:** Observed in the Extra-Small (XS) scale tests, where $R_{mem} = 1.10777$. In this regime, the memory required for SRF's internal management metadata, fixed-size double-buffers, and checkpoint indices exceeds the memory that would be required by a naive quadratic-space implementation. This identifies the scale below which the recomputation-based execution model does not provide a physical memory benefit. The management floor is a function of the checkpoint density and the granularity of the internal tracking structures, defining the minimum "profitable" input scale for recomputation.
2. **Recomputation Domination:** This state was observed in the 250,000-node Graph-DP extreme-scale test. The recompute-to-compute ratio indicated that the framework spent a significant portion of its duty cycle reconstructing node values rather than advancing the topological evaluation. This state, formally classified as `RECOMPUTATION_DOMINATED`, occurs when the graph density or the recomputation depth setting causes the restoration kernels to outweigh the forward progress of the algorithm. This regime identifies the point where the compute penalty of memory reduction begins to limit the practical utility of the framework for real-time analysis.

To formalize the detection of the management floor, we define the memory consumption of the SRF implementation as $M_{SRF}(N) = M_{meta} + cN$, where $M_{meta}$ is the constant overhead of the management infrastructure and $cN$ is the linearly

scaling buffer requirement. The memory consumption of a classical baseline implementation is $M_{base}(N) = c'N^2$. The management floor occurs at the input scale $N_{min}$ where $M_{SRF}(N_{min}) = M_{base}(N_{min})$. Conceptually, this intersection defines the boundary of the memory-efficiency regime. For $N < N_{min}$, the quadratic term is smaller than the combined constant and linear overheads of the recomputation framework. As $N$ increases beyond this point, the quadratic growth of $M_{base}$ quickly outpaces the linear growth of $M_{SRF}$, leading to the $> 99\%$ savings observed at genomic scales. The empirical measurement of $R_{mem} > 1$ at the XS scale provides a direct observation of this crossover point, verifying that the benefits of recomputation are restricted to workloads where the theoretical complexity bottleneck manifests as a physical resource constraint.

Importantly, no locality collapse or hardware-level memory exhaustion was observed during the memory-constrained tests, as the core C++ binaries successfully completed their recomputation schedules within the artificially imposed 256MB ulimit. These observations define the stable operational envelope of the framework across the tested biological datasets and scales, providing an empirical baseline for the framework's stability claims.

## 4 Discussion

### 4.1 Relationship to Linear-Space Divide-and-Conquer

The execution model implemented in the Structured Recomputation Framework (SRF) bears a fundamental relationship to the classical linear-space algorithms for sequence alignment, most notably the work of Hirschberg [15] and the subsequent adaptation for affine gap penalties by Myers and Miller [17]. Hirschberg's algorithm introduced the principle of using divide-and-conquer to trade computational time for memory space, reducing the quadratic storage requirement of the Needleman-Wunsch algorithm to linear space. SRF operates at a different level of abstraction by formalizing this trade-off as a general execution-level restructuring rather than a modification of a specific algorithm. While Hirschberg's method relies on a recursive split-point identification strategy that essentially doubles the total computation, SRF utilizes an iterative, unit-based recomputation schedule that can be tuned via granularity parameters to balance the overhead more precisely against the available physical memory.

In the context of affine gap penalties, Myers and Miller demonstrated that the linear-space benefit could be maintained by carefully managing the dependencies between the match, insertion, and deletion matrices. SRF instantiates these theoretical bounds through its triple-buffered recomputation tiles in SRF-Needleman-Wunsch. The framework's ability to achieve $O(N)$ space through tiling and double-buffering represents a practical engineering instantiation of the same information-theoretic constraints. Unlike the specialized recursive structure of Hirschberg's method, SRF maintains the original algorithmic recurrence relations in their exact, non-recursive forms, allowing for an implementation that is closer to the original DP definition. This iterative approach simplifies the management of the execution frontier and allows for the application of low-level architectural optimizations, such as SIMD vectorization of the cell calculations, which are often more complex to integrate into highly recursive divide-and-conquer structures. The systematic mapping of

state matrices to tile boundaries provides a robust mechanism for ensuring that gap extension logic is perfectly preserved across recomputation events.

### 4.2 Relationship to Checkpointing and Recomputation Literature

The SRF architecture is grounded in the broader literature of algorithmic checkpointing and automatic differentiation. The formalization of recomputation schedules was significantly advanced by Griewank [28, 29], who established the logarithmic and square-root relationships between the number of stored checkpoints and the total computational cost in the context of reverse-mode differentiation. SRF instantiates these models within the domain of biological dynamic programming, providing a unified engine for managing recomputation across diverse algorithm classes. The framework's `RecomputeScheduler` essentially solves a localized version of the checkpointing schedule problem, identifying the minimal set of boundary states required to restore a given computation unit. This formalization allows the framework to dynamically adapt its recomputation strategy based on the detected memory pressure, ensuring that the overhead remains within predictable bounds.

In the context of profile HMMs and sequence-to-profile alignment, specialized checkpointing schemes have long been employed in tools like HMMER to manage the storage of the trellis for posterior decoding [19]. SRF unifies these disparate, algorithm-specific recomputation strategies into a single structural abstraction. By decoupling the checkpointing interval from the specific state transitions of the HMM or the grid structure of the alignment matrix, the framework allows for a standardized approach to memory management. This unification ensures that optimizations such as cache-aligned tiling and deterministic state restoration are applied consistently, regardless of whether the underlying algorithm is an HMM evaluator or a DAG-based propagation kernel. The framework's internal management of state eviction and unit-based restoration automates the trade-off that was previously handled through manual, algorithm-specific interventions in specialized bioinformatics tools, providing a more scalable and maintainable foundation for memory-efficient computing. This abstraction layer ensures that new algorithms can benefit from established recomputation patterns without requiring low-level memory management code.

### 4.3 Execution Abstraction Versus Algorithm-Specific Checkpointing

A critical distinction exists between algorithm-specific checkpointing and the unified execution abstraction proposed in this work. Formally, algorithm-specific checkpointing is characterized by a recomputation logic that is tightly coupled to the internal state transitions and mathematical structure of a specific recurrence. In sequence alignment, this is manifested in the recursive pivot selection of the Hirschberg method, where the memory-saving split is integrated directly into the sequence-matching logic. Similarly, traditional HMM segment checkpointing often relies on predefined biological boundaries or specific trellis properties to manage state persistence. In these instances, the recomputation strategy is not an independent layer but an integral part of the algorithm's mathematical definition.

SRF, conversely, is implemented as a recurrence-agnostic execution abstraction. It does not introduce a new biological recurrence or alter the fundamental mathematical logic of sequence alignment or probabilistic modeling. Instead, SRF formalizes

execution restructuring at the directed acyclic graph (DAG) level, treating every dynamic programming state as a node in a dependency network. This allows for a clean separation between the algorithmic redesign—which modifies the biological recurrence—and the execution schedule transformation, which only alters the physical timing and persistence of state evaluations. Consequently, SRF does not claim to introduce new asymptotic bounds beyond those established in the recomputation literature; its value lies in providing a stable and deterministic engine that can lower any DP DAG into a memory-bounded execution frontier. This abstraction enables the framework to generalize optimizations that were previously isolated to specific sequence-based tools, applying them uniformly to diverse problems such as graph-based score propagation and multi-state HMM trellis evaluations.

### 4.4 Relationship to Cache-Aware and Cache-Oblivious Dynamic Programming

The performance characteristics of SRF are influenced by the interaction between the recomputation schedule and the computer's memory hierarchy. The work of Frigo et al. [22] on cache-oblivious algorithms demonstrated that recursive structures can be designed to optimize for any cache hierarchy without explicit parameter tuning. Similarly, Chowdhury and Ramachandran [23] developed cache-efficient versions of dynamic programming algorithms that minimize memory bus traffic. These techniques focus on improving the temporal and spatial locality of memory accesses to overcome the "memory wall" bottleneck that often limits the performance of large-scale DP computations.

SRF operates at the level of execution restructuring, utilizing granularity parameters to align recomputation units with physical hardware constraints. While cache-oblivious algorithms focus on asymptotic optimality across all cache levels, SRF provides a more explicit control surface for the user to define hard memory bounds. The recomputation tiles in SRF-Needleman-Wunsch are sized such that the restoration of a tile occurs within the higher levels of the processor's cache (L1/L2), thereby mitigating the latency of the additional compute cycles. By ensuring that the working set of a recomputation kernel fits within the local cache, the framework reduces the frequency of expensive main-memory fetches and minimizes the probability of cache line evictions during critical path reconstruction steps. The framework does not claim architectural optimality but rather provides a practical mechanism for balancing the bandwidth-limited costs of main memory access against the relatively abundant compute cycles available in modern multicore processors. This architectural awareness is essential for maintaining predictable execution times even as input sizes scale to genomic dimensions, ensuring that the framework remains efficient on diverse hardware.

### 4.5 Relationship to Theoretical Time–Space Tradeoffs

The trade-off between time and space is a central theme in computational complexity theory. A recent theoretical result by Ryan Williams [30] in "Simulating Time With Square-Root Space" provides a rigorous foundation for understanding the limits of how much memory can be saved for a given increase in execution time. This work explores the simulation of time-bounded computations using significantly less space than the time bound would traditionally suggest, establishing a new theoretical frontier for time-space simulations in deterministic environments.

SRF operates at the level of practical algorithm engineering and does not implement or extend the specific theorems presented by Williams. The framework is an engineering-level instantiation of known time–space tradeoffs rather than a theoretical proof of new complexity class results. No claims of asymptotic improvement are made regarding the underlying complexity of the DP algorithms. However, the theoretical context provided by such work is valuable for framing the empirical results of the framework. The SRF execution model is a concrete instantiation of the principle that many dynamic programming problems—which are inherently time-intensive—can be restructured to run in reduced space through controlled recomputation. The framework provides a bridge between these high-level complexity-theoretic bounds and the physical constraints of genomic data processing, focusing on the engineering stability and deterministic reproducibility of the results. The observed stability of $R_{mem}$ and $R_{rec}$ across different hardware environments confirms that the engineering practice of structured recomputation is a viable strategy for handling the massive datasets encountered in modern molecular biology, allowing space-efficiency gains to be realized in practical, production-grade software that must operate under real-world constraints.

### 4.6 Practical Scope and Limitations

The evaluation of SRF has identified several important constraints on its practical utility. First, the framework does not alter the fundamental quadratic time complexity of algorithms such as Needleman-Wunsch. The reduction in memory footprint is achieved through the introduction of a recomputation overhead, which was measured at approximately 47% for biological mitochondrial DNA alignments at the XS scale. This overhead is a necessary consequence of the functional recomputation model and means that the total execution time will always be greater than or equal to a single-pass, high-memory implementation. The recompute penalty is a deterministic cost that must be weighed against the benefit of being able to process larger datasets on limited hardware.

The benefit of using SRF is directly dependent on the memory constraints of the execution environment. In scenarios where sufficient RAM is available to store the entire DP matrix, the framework's recomputation overhead provides no performance advantage and may even be counterproductive due to the added management logic and state tracking structures. Furthermore, the management of checkpoints and the maintenance of the snapshots introduce a small but measurable management floor, below which the memory overhead of the framework's infrastructure exceeds the storage requirements of the baseline algorithm. For sequences shorter than 500 bp, the framework exhibited an $R_{mem}$ of 1.10777, indicating that the metadata and buffer management exceeded the baseline memory cost. SRF is not intended to replace heuristic search tools like BLAST, which achieve speedups by sacrificing exactness. Instead, it is designed for scenarios where exact path reconstruction or posterior decoding is required but prohibited by physical memory limits on standard commodity hardware. The adaptive mechanisms also introduce a minor computational overhead for drift detection, which is negligible for large-scale workloads but observable in micro-benchmarks. These limitations define the operational envelope within which the framework provides its primary scientific value.

## 5 Conclusions

The Structured Recomputation Framework (SRF) demonstrates a stable and deterministic execution model for dynamic programming in bioinformatics. By restructuring the execution of foundational algorithms such as sequence alignment, HMM trellis evaluation, and graph-based propagation, the framework enables the processing of large-scale biological datasets within strictly bounded memory envelopes. The core of this approach is the systematic use of hierarchical checkpointing and on-demand recomputation to trade abundant compute cycles for scarce memory space. This execution-level transformation preserves the exact semantics of the underlying algorithms, ensuring that the results of genomic analyses are indistinguishable from those produced by traditional implementations. The stability of the framework under artificially imposed memory constraints verifies the robustness of the state eviction policies and the reliability of the restoration kernels in processing diverse biological sequences.

The empirical characterization of the framework across diverse algorithmic paradigms has verified its structural invariance. The recomputation ratios and memory intensity metrics remained bit-exact across different compilers and optimization levels, confirming the stability of the execution schedule. This deterministic behavior is a critical property for scientific reproducibility, particularly in multi-platform environments where compiler variations or hardware differences could otherwise introduce numerical drift in probability calculations. The identification of analytical regimes, such as the `MEMORY_BOUND` and `COMPUTE_BOUND` states, provides a formal vocabulary for describing the operational limits of recomputation-based systems. These regimes allow for the objective quantification of the efficiency of an algorithm's execution and inform the adaptive tuning of granularity parameters to satisfy specific hardware constraints, ensuring that resource usage remains aligned with the available physical capacity.

The implementation of the framework in standard C++17 with a backend abstraction layer ensures that the benefits of recomputation are accessible across a wide range of hardware, from local workstations to high-performance computing clusters. The separation of the algorithmic recurrence from the execution schedule allows for the application of architectural optimizations and memory-reduction techniques without requiring the manual re-derivation of the DP logic for each new problem domain. This unified abstraction democratizes the use of high-sensitivity, memory-intensive algorithms, making it possible to perform exact genome-scale analyses on commodity hardware that would otherwise be restricted to high-memory server configurations. The ability to execute exact alignments and posterior decodings on standard hardware reduces the dependence on specialized infrastructure and provides a more flexible foundation for genomic research.

The framework also provides a stable baseline for the future development of cloud-based and distributed bioinformatics pipelines. By strictly bounding the memory footprint of individual tasks, SRF allows for more efficient task scheduling and higher density of concurrent evaluations on cloud instances. This predictability is essential for managing the cost and stability of large-scale genomic workflows. Furthermore, the deterministic nature of the recomputation kernels ensures that tasks can be resumed or re-executed across different nodes without the risk of numerical

divergence, facilitating more robust distributed computing patterns. This ability to manage the time-space trade-off at the execution level provides a significant advantage for processing the massive, distributed datasets generated by global sequencing efforts.

In conclusion, this work provides a unified abstraction for memory-constrained algorithmic engineering in molecular biology. While the recomputation overhead represents a predictable and measurable cost, the resulting ability to execute exact genomic analyses within strictly controlled memory boundaries provides a reliable foundation for the development of the next generation of bioinformatics tools. The framework establishes that functional recomputation is not only a theoretical possibility but a practical and reproducible engineering solution for the memory scaling challenges of modern computational biology. By providing a stable execution engine that bridges the gap between mathematical recurrences and physical hardware limits, SRF enables more comprehensive and sensitive analyses of the massive datasets generated by modern sequencing technologies, ensuring that computational capacity remains aligned with the growing scale of biological information and facilitating new insights into the molecular foundations of life.

**References**


1. Needleman, S.B., Wunsch, C.D.: A general method applicable to the search for similarities in the amino acid sequence of two proteins. Journal of Molecular Biology **48**(3), 443–453 (1970)
2. Smith, T.F., Waterman, M.S.: Identification of common molecular subsequences. Journal of Molecular Biology **147**(1), 195–197 (1981)
3. Durbin, R., Eddy, S.R., Krogh, A., Mitchison, G.: Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids. Cambridge University Press, Cambridge (1998)
4. Rabiner, L.R.: A tutorial on hidden markov models and selected applications in speech recognition. Proceedings of the IEEE **77**(2), 257–286 (1989)
5. Krogh, A., Brown, M., Mian, I.S., Sjölander, K., Haussler, D.: Hidden markov models in computational biology: Applications to protein modeling. Journal of Molecular Biology **235**(5), 1501–1531 (1994)
6. Eddy, S.R.: Profile hidden markov models. Bioinformatics **14**(9), 755–763 (1998)
7. Altschul, S.F., Gish, W., Miller, W., Myers, E.W., Lipman, D.J.: Basic local alignment search tool. Journal of Molecular Biology **215**(3), 403–410 (1990)
8. Lipman, D.J., Pearson, W.R.: Rapid and sensitive protein similarity searches. Science **227**(4693), 1435–1441 (1985)
9. Pearson, W.R., Lipman, D.J.: Improved tools for biological sequence comparison. Proceedings of the National Academy of Sciences **85**(8), 2444–2448 (1988)
10. Sellers, P.H.: On the theory and computation of evolutionary distances. SIAM Journal on Applied Mathematics **26**(4), 787–793 (1974)
11. Waterman, M.S.: Introduction to Computational Biology: Maps, Sequences and Genomes. CRC Press, Boca Raton (1995)
12. Gotoh, O.: An improved algorithm for matching biological sequences. Journal of Molecular Biology **162**(3), 705–708 (1982)
13. Gusfield, D.: Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology. Cambridge University Press, Cambridge (1997)
14. Apostolico, A.: The ubiquitous digital tree. Pattern Matching Algorithms, 1–21 (1985)
15. Hirschberg, D.S.: A linear space algorithm for computing maximal common subsequences. Communications of the ACM **18**(6), 341–343 (1975)
16. Park, K., Kim, D.K.: A speed-up of hirschberg's algorithm for longest common subsequences. Information Processing Letters **53**(6), 345–348 (1995)
17. Myers, E.W., Miller, W.: Optimal alignments in linear space. Bioinformatics **4**(1), 11–17 (1988)
18. Zhang, Z., Berman, P., Miller, W.: Aligning alignments: An algorithm for global alignment of sequences with gap penalties. Bioinformatics **16**(1), 33–40 (2000)
19. Eddy, S.R.: Accelerated profile hmm searches. PLOS Computational Biology **7**(10), 1002195 (2011)
20. Gotoh, O.: Alignment of three biological sequences with an efficient dynamic programming algorithm. Journal of Theoretical Biology **121**(3), 327–340 (1986)
21. Tan, G., Sun, N., Gao, G.R.: Improving the performance of needleman-wunsch algorithm on modern multi-core architectures. Proceedings of the 2006 International Conference on Parallel Processing (ICPP), 115–122 (2006)
22. Frigo, M., Leiserson, C.E., Prokop, H., Ramachandran, S.: Cache-oblivious algorithms, 285–297 (1999)
23. Chowdhury, R.A., Ramachandran, V.: Cache-efficient dynamic programming algorithms, 1129–1138 (2006)
24. Wozniak, A.: Using video-oriented instructions to speed up sequence comparison. Bioinformatics **13**(2), 145–150 (1997)
25. Rognes, T.: Faster smith-waterman database searching with inter-sequence simd parallelisation. BMC Bioinformatics **12**(1), 221 (2011)
26. Daily, J.: Parasail: Simd c library for global, semi-global, and local pairwise sequence alignment. BMC Bioinformatics **17**(1), 1–11 (2016)
27. Farrar, M.: Striped smith-waterman speeds database searches six times over other simd implementations. Bioinformatics **23**(2), 156–161 (2007)
28. Griewank, A.: Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation. Optimization Methods and Software **1**(1), 35–54 (1992)
29. Griewank, A., Walther, A.: Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation. SIAM, Philadelphia (2000)
30. Williams, R.: Simulating time with square-root space. ACM Symposium on Theory of Computing (STOC) (2025). Preprint/Accepted Manuscript


**List of abbreviations**

SRF: Structured Recomputation Framework; DP: Dynamic Programming; HMM: Hidden Markov Model; MSA: Multiple Sequence Alignment; CP: Checkpoint.

**Ethics approval and consent to participate**

Not applicable.

**Consent for publication**

Not applicable.

**Availability of data and materials**

The Structured Recomputation Framework (SRF) reference implementation and all benchmark pipelines are available at the following repository:
https://github.com/Sulkysubject37/SRF
The repository contains the full C++17 implementation, benchmarking scripts, continuous integration workflows, and datasets used in this study.

**Competing interests**
The author declares that he has no competing interests.

**Funding**
Not applicable.

**Authors' contributions**
MA conceptualized the SRF model, implemented the framework, performed the validation, and wrote the manuscript.

**Authors' information**
MD. Arshad is a researcher in the Department of Computer Science at Jamia Millia Islamia, New Delhi, India. His work focuses on high-performance algorithmic engineering and deterministic computing in computational biology.

**Acknowledgements**
Not applicable.